

Processos e comunicação entre processos

fork, memória independente e pipe na prática

Weslei Ferreira Santos

5 de outubro de 2026

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

min	Etapa
0–8	Programa e processo
8–20	Uma chamada de <code>fork()</code>
20–33	<code>fork()</code> dentro do laço: <code>fork2.c</code>
33–45	Memória independente e <code>waitpid</code>
45–53	Checagem em duplas
53–65	Pipe: canal, pontas e bloqueio
65–75	Aplicação com vários processos: <code>pipe2.c</code>
75–87	Prática: filho envia, pai recebe
87–90	Bilhete de saída

Como um processo recebe o resultado calculado por outro?

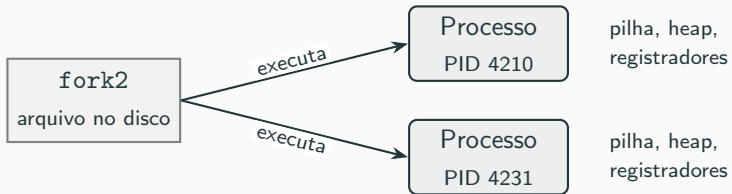
Ao final, você deve conseguir:

- identificar os retornos de `fork()` no pai e no filho;
- explicar por que cada processo tem a sua própria memória;
- desenhar a árvore de processos de um laço com `fork()`;
- completar uma comunicação filho → pai com `pipe()`.

Programa e processo

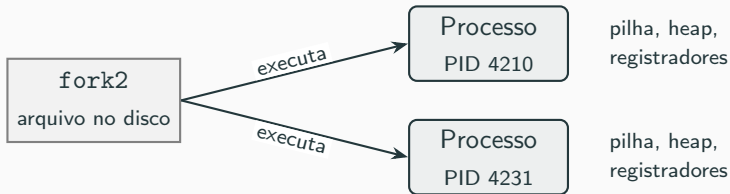
Um programa ou dois processos?

Se eu executar o mesmo programa em dois terminais, tenho um programa ou dois processos?



Um programa ou dois processos?

Se eu executar o mesmo programa em dois terminais, tenho um programa ou dois processos?



Definição

O arquivo contém as instruções. Cada execução é um **processo**: uma instância com PID, registradores, pilha, heap e recursos próprios.

Se uma dessas execuções altera uma variável,
a outra *necessariamente* observa a alteração?

Anote sua hipótese. Vamos testá-la com `fork2.c`.

Uma chamada de `fork()`

Uma chamada, dois caminhos

```
pid_t pid = fork();
if (pid == -1) {
    /* falha: nenhum filho criado */
} else if (pid == 0) {
    /* este caminho executa no filho */
} else {
    /* executa no pai; pid = filho */
}
/* pai e filho podem chegar aqui */
```

Quem	fork() retorna
Filho	0
Pai	PID do filho
Falha	-1

O filho não recomeça o main

Pai e filho continuam **depois** da chamada, cada um com seu retorno.

getpid() e a ordem de execução

- getpid() retorna o PID do processo que *está executando*.
- O valor que fork() devolve ao pai identifica o **filho**, não o próprio pai.

Previsão: o pai necessariamente executa o próximo printf antes do filho?

getpid() e a ordem de execução

- getpid() retorna o PID do processo que *está executando*.
- O valor que fork() devolve ao pai identifica o **filho**, não o próprio pai.

Previsão: o pai necessariamente executa o próximo printf antes do filho?

Resposta

Não. A ordem depende do escalonamento e da sincronização que o programa estabelecer.

`fork()` dentro do laço

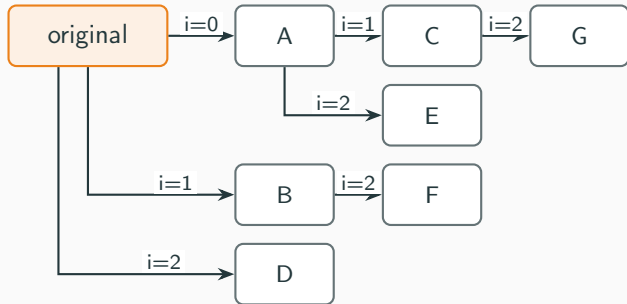
fork2.c: o trecho central

```
int *v = calloc((size_t)tamanho, sizeof *v);    /* tamanho = 3 */
pid_t original = getpid();
for (int i = 0; i < tamanho; ++i) {
    pid_t pid = fork();
    if (pid == -1) { /* tratamento de falha */ }
    if (pid == 0) v[i] = i + 5;
}
```

Dois minutos, no papel

Com tamanho = 3, quantos processos existem ao todo? Desenhe quem cria quem.

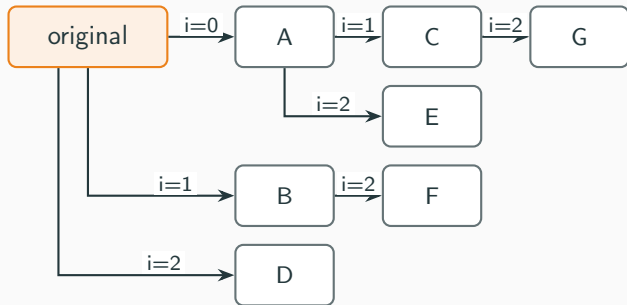
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1

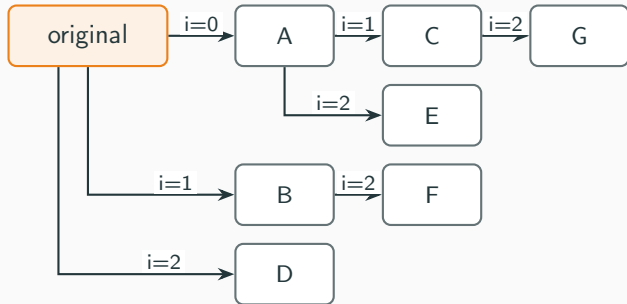
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- $i=0$: 2

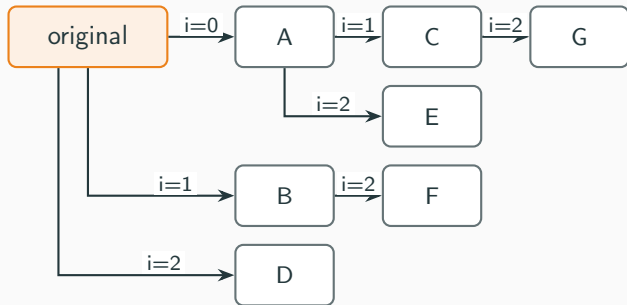
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- $i=0$: 2
- $i=1$: 4

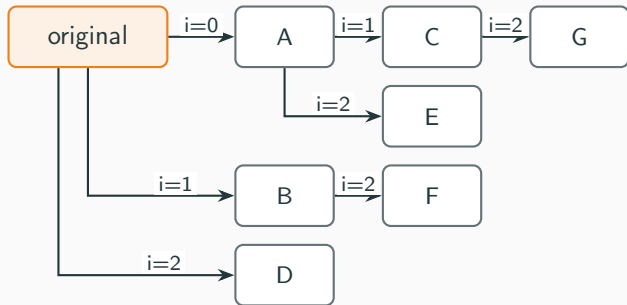
A árvore de fork2.c



Caminhos ao fim de cada
iteração

- início: 1
- $i=0$: 2
- $i=1$: 4
- $i=2$: 8

A árvore de fork2.c



Caminhos ao fim de cada iteração

- início: 1
- i=0: 2
- i=1: 4
- i=2: 8

Todos continuam o laço: **8 processos** ao todo, **7 novos**,
 $1 + 2 + 4 = 7$ chamadas executadas.

Dois cuidados com essa contagem

- **8** conta os processos criados *ao longo da execução*.
Não significa que os oito estejam vivos ao mesmo tempo.
- O código tem **uma** chamada de `fork()`; a árvore executa **sete**.

E se cada filho saísse do laço logo após nascer?

Dois cuidados com essa contagem

- **8** conta os processos criados *ao longo da execução*.
Não significa que os oito estejam vivos ao mesmo tempo.
- O código tem **uma** chamada de `fork()`; a árvore executa **sete**.

E se cada filho saísse do laço logo após nascer?

Resposta

Só o original completaria as três iterações: **4 processos** ao todo (o original e três filhos).

Memória independente

Previsão antes de executar

Cada filho escreve em $v[i]$. O pai soma o vetor no final.

Qual soma o pai imprime? 0? 18? outro valor?

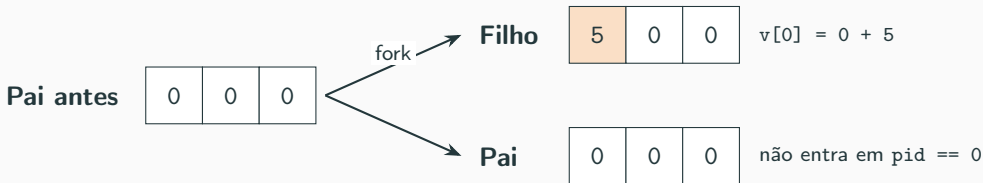
Previsão antes de executar

Cada filho escreve em `v[i]`. O pai soma o vetor no final.

Qual soma o pai imprime? 0? 18? outro valor?

```
$ make run-fork  
./build/fork2  
Soma total no pai: 0
```

Cada processo escreve na própria cópia



- `calloc` começa com três zeros.
- O filho altera a **sua** memória; o vetor do pai continua `[0, 0, 0]`.
- `getpid() == original`: só o processo original imprime a soma.

waitpid: esperar não é comunicar

O que waitpid faz

- Espera um filho terminar.
- Coleta o estado de término.
- Cada processo coleta seus **filhos diretos**.

O que ele não faz

- Não copia o vetor do filho.
- Não cria memória compartilhada.
- Não transporta resultados.

Por que a soma não virou 18?

O pai já espera os filhos, mas esperar coordena o *término*. Nem sleep nem waitpid tornam o heap compartilhado.

1. Quantos processos são criados ao todo, incluindo o original? Desenhe a evolução por iteração.
2. Quais retornos de `fork()` identificam filho, pai e falha? De onde cada um continua?
3. O filho altera `v[i]`. Qual soma o pai imprime? Explique.
4. O pai espera os filhos. Isso faz o vetor dele receber as escritas dos filhos?

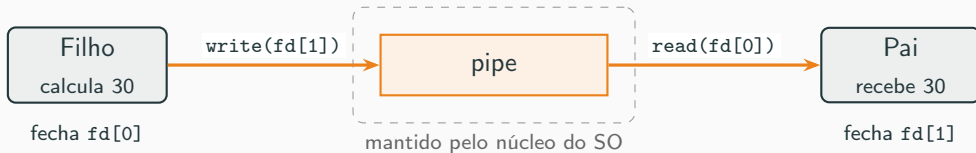
Vamos ouvir uma dupla sobre a árvore e outra sobre a espera.

Pipe: comunicação explícita

Se a memória não leva o resultado...

...o filho precisa **enviar** os dados explicitamente.

IPC (*Inter-Process Communication*): mecanismos do SO para processos trocarem dados.



Pontas, descritores e herança

- `pipe(fd)` cria o canal: `fd[0]` é a ponta de **leitura**, `fd[1]` a de **escrita**.
- **Descritor**: número que identifica um recurso aberto pelo processo.
- O pipe é criado **antes** do `fork()`: pai e filho herdam descritores que referenciam o mesmo canal.
- Cada processo fecha a ponta que não usa.

Ordem que importa

`pipe()` → `fork()` → cada um fecha uma ponta → `write` / `read` → `waitpid`

O exemplo pequeno: pipe_soma.c

```
if (pid == 0) {                                /* filho */
    close(fd[0]);
    valor = 10 + 20;
    int resultado = transferir(fd[1], &valor, sizeof valor, 1);
    close(fd[1]);
    _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
close(fd[1]);                                  /* pai */
int resultado = transferir(fd[0], &valor, sizeof valor, 0);
close(fd[0]);
```

transferir chama write (modo 1) ou read (modo 0) até mover todos os bytes.

O mesmo &valor, duas variáveis

- &valor no filho e &valor no pai apontam para variáveis **locais de cada processo**.
- O SO **copia os bytes** pelo canal.
- O ponteiro não torna as duas variáveis compartilhadas: o endereço é interpretado dentro do espaço de cada processo.

Pipe é fluxo de bytes

Nem toda chamada de read recebe tudo que pedimos. O *helper* transferir repete a operação e trata interrupções (EINTR).

E se o pai chegar ao read primeiro?

Situação do pipe	O que read faz
Vazio, com ponta de escrita aberta	Bloqueia e espera dados
Com dados disponíveis	Retorna os bytes (talvez menos que o pedido)
Vazio, todas as escritas fechadas	Retorna 0: EOF

Por isso fechamos pontas

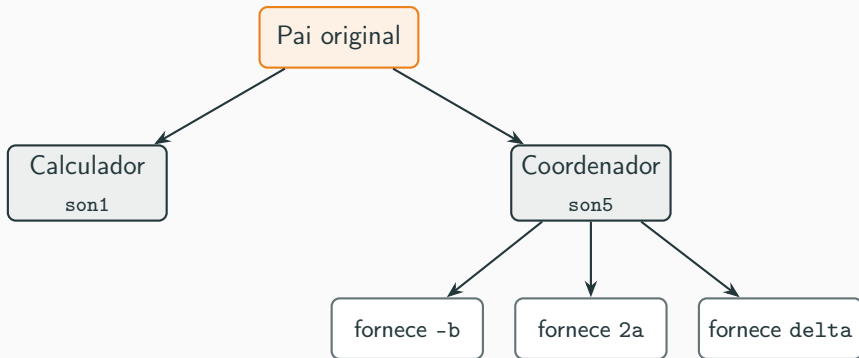
Enquanto houver *qualquer* ponta de escrita aberta, inclusive uma cópia herdada esquecida, um pipe vazio não sinaliza EOF.

Executando o exemplo

```
$ make run-pipe-soma  
./build/pipe-soma  
Resultado recebido: 30
```

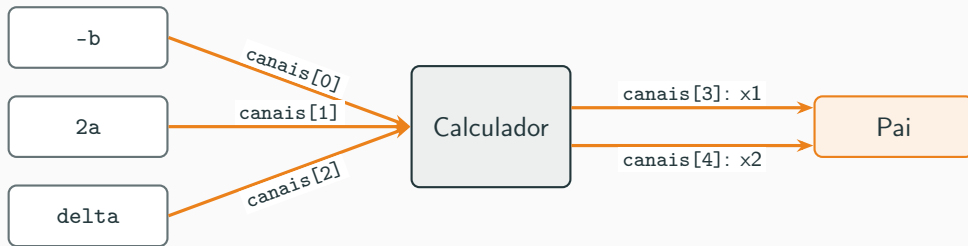
- O 30 foi calculado no filho.
- Chegou ao pai pelo canal, não por memória comum.

Vários processos: pipe2.c



Seis processos e cinco pipes. A entrada é lida no pai antes dos `fork()`.

pipe2.c: por onde os dados passam



Árvore de criação $\neq$ fluxo de dados

Os fornecedores são filhos do coordenador, mas enviam ao calculador. Quem escreve e quem lê cada canal?

Executando pipe2.c

```
$ make run-pipe  
Digite a, b e delta (delta ja calculado): 1 -3 1  
Raizes: x1 = 1; x2 = 2
```

Entrada: a, b e delta.

O programa **não** pede c.

$$-b = 3, \quad 2a = 2, \quad \sqrt{\Delta} = 1$$

$$x_1 = \frac{3-1}{2} = 1, \quad x_2 = \frac{3+1}{2} = 2$$

O exemplo ilustra IPC; não foi feito para acelerar um cálculo tão pequeno.

Prática

Seis lacunas: pipe_soma_atividade.c

```
if (pid == 0) {
    close(/* 1: ponta que o filho nao usa */);
    valor = /* 2: calculo */;
    int resultado = /* 3: enviar valor pelo helper transferir */;
    close(fd[1]);
    _exit(resultado == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}
close(/* 4: ponta que o pai nao usa */);
int resultado = /* 5: receber valor pelo helper transferir */;
close(/* 6: ponta usada pelo pai */);
```

transferir(descriptor, endereço, bytes, modo) — modo 1 envia, 0 recebe.

Compilar e conferir

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \  
    examples/dia05/pipe_soma_atividade.c -o /tmp/pipe-soma-aluno  
$ /tmp/pipe-soma-aluno  
Resultado recebido: 30
```

Em duplas, 7 minutos

- Antes de preencher, o esqueleto não compila.
- Cada pessoa explica uma ponta do canal.

Responda também

5. Por que criar o pipe antes do `fork()`?
6. `read` antes do `write`: erro, espera ou EOF?
7. Por que cada um fecha uma ponta diferente?

Fechamento

<code>fork()</code>	Cria um processo com memória independente .
<code>waitpid()</code>	Coordena o término e coleta o filho.
<code>pipe()</code>	Transporta dados entre processos.

Próxima aula: 7/10

Threads **compartilham** memória. O problema muda: como controlar o acesso a ela?

Individual, em duas ou três frases:

Por que `waitpid` não resolve a comunicação do resultado e `pipe` resolve?

Dúvidas?

Weslei Ferreira Santos
MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Material-base: Prof. Maycon Leone M. Peixoto